Development of RentaMovil: An Inventory Management and Fleet Control System Based on Hexagonal Architecture and Quality Metrics

Nicole Dayana Ramirez Vargas
Servicio Nacional de Aprendizaje – SENA
Neiva, Colombia
nicolerv18007@gmail.com

Thiago Fabian Rojas Guevara
Servicio Nacional de Aprendizaje – SENA
Neiva, Colombia
thiagosrojasg451@gmail.com

David Santiago Erez Rodriguez
Servicio Nacional de Aprendizaje – SENA
Neiva, Colombia
erezsantiago25@gmail.com

1. Introduction

1.1 The Problem

In small and medium-sized businesses, vehicle rental is usually handled with whatever is at hand: a spreadsheet for the fleet, a paper notebook or a chat for reservations, and memory for payments and maintenance. The system holds up for a while. Then it breaks.

That is precisely what we found: operational disorder. Nobody could say with certainty which vehicles were available on a given date. Reservations overlapped one another, because there was no single point that validated availability. Payments were taken as good without a verifiable receipt, and maintenance arrived late, always reactive, when the vehicle had already failed. On top of that came two more gaps: there was no tracking of vehicles during the rental, and generating a formal document — an invoice, for example — was impossible.

There is an idea that software engineering repeats to exhaustion and that applies fully here: the quality of the product depends, to a large extent, on the quality of its design and of the practices with which it is built [Sommerville 2015]. It was not enough for the system to work. We needed its structure to be understandable, maintainable and verifiable. The problems, specifically, were several: scattered, uncentralized information; a reservation cycle impossible to trace from beginning to end; reactive maintenance with no preventive planning; availability conflicts due to the lack of a single source of truth about which vehicle was occupied; and nonexistent access control, with no roles or permissions to separate the customer, the administrator and the superadministrator.

1.2 Research Motivation

The underlying motivation was not just to solve the rental problem. We wanted the system to serve as a case study: to demonstrate that a disciplined design yields measurable and maintainable software. That is why the team did the opposite of ”jumping straight into coding.” It documented the design before implementing it (SDD), modeled the business with DDD, and distributed responsibilities with hexagonal architecture and microservices [Evans 2003; Cockburn 2005; Newman 2021].

There was a second purpose. From the start we defined quality indicators to evaluate, with some objectivity, the progress and soundness of the system. The DevOps literature points in this direction: teams that measure their performance systematically tend to obtain better results [Forsgren et al. 2018; Forsgren 2021]. We found this to be a verifiable claim and wanted to test it in a real academic project, limitations and all.

1.3 Research Objectives

The general objective of this research was to develop a vehicle rental system that left behind the limitations of the manual process, making use of good architecture practices and with the quality of the result measured, not assumed. From this, six specific objectives followed:

- Model the business domain and identify bounded contexts with DDD.
- Implement microservices with an API Gateway and an internal hexagonal architecture.
- Establish a normalized data model, with the schema under version control.
- Secure the system with JWT tokens, roles and permissions.
- Build the web and mobile clients on documented API contracts.
- Define and evaluate a set of quality indicators.

1.4 Importance of the Topic

Efficient management of the fleet and reservations is not a luxury: it is competitiveness. Automating these processes pushes reservation, payment and delivery to be reliable and profitable. In the Colombian case, adopting good engineering and technology management practices bears directly on the productivity of companies [Espinosa and Eraso 2024]. RentaMovil puts these principles to the test in the mobility sector, and its documentation remains as a reference for any company that wants to attempt something similar.

2. Related Work

This section reviews the six fronts that underpin the design of RentaMovil: microservices, hexagonal architecture, domain-driven design, quality metrics, design patterns and web security.

2.1 Microservices Architecture

For Newman [Newman 2021], microservices are a style that splits the application into small, separately deployable services organized around business capabilities. [Fowler and Lewis 2014] add the other side of the scale: the style gains functional cohesion and team autonomy,

but it charges its price in operations and in communication between services, which has to be managed. This is where Richardson's patterns [Richardson 2018] come in — API Gateway, database per service, idempotent consumers, event publishing — designed precisely to reduce that cost. RentaMovil adopts several of them: single entry points, a database schema per service, and idempotent event consumers.

Piñero González et al. [Piñero González et al. 2021] identify that performance efficiency in software products is related to

2.2 Hexagonal Architecture (Ports and Adapters)

Cockburn [Cockburn 2005] devised hexagonal architecture for something simple and difficult at once: isolating the business from the technology. The domain declares ports (interfaces); the infrastructure provides adapters that plug them into databases, web services or messaging. Martin [Martin 2017] takes it to a rule: dependencies point toward the domain, never the other way around, and that is dependency inversion. In RentaMovil each microservice respects the separation — the domain packages know nothing about Spring or JPA — and not out of dogma, but because it spares us pain when something external changes.

2.3 Domain-Driven Design (DDD)

Evans [Evans 2003] left three concepts that we use almost daily: ubiquitous language, bounded context and context map. The idea is for the software to speak the language of the business. In RentaMovil the boundaries of each service come from their bounded contexts (Identity, Fleet, Reservations, Rental, Payments, Telemetry and Notification). Decoupling, thus, rests on the domain and not on the technology fad of the moment.

2.4 Quality Metrics and DevOps

Forsgren, Humble and Kim [Forsgren et al. 2018] did not stop at theory: they showed with data that continuous delivery and a decoupled architecture are better predictors of performance. Their DORA metrics — deployment frequency, lead time, failure rate and recovery time — ended up becoming an industry standard [Forsgren 2021]. Humble and Farley insist on two things which we take for granted, but in reality cannot set aside: automation and fast feedback. We took those ideas and adapted them into an engineering KPI dashboard tailored to an academic project.

2.5 Design Patterns and Layer Separation

The classic patterns [Gamma et al. 1994] and those of enterprise architecture [Fowler 2002] are already explicit solutions to common, repetitive problems: Repository used for data access, Data Transfer Object for transferring information between layers, and Service Layer for isolating business logic. Parnas [Parnas 1972] had said it long before: a good division — one that hides the design decisions that are going to change — is what sustains maintainability. RentaMovil applies them without inventing anything.

2.6 Security in Web Applications

A system that handles personal data and payment transfers cannot treat the security of these sensitive data with such looseness and lightness. OWASP [OWASP 2021] enumerates the 10 most critical security vulnerabilities: broken authentication, sensitive data exposure, access control failures, injections, etc. How RentaMovil responds to these vulnerabilities combines signed JWTs, password hashing with BCrypt, permission-based authorization, input validation and parameterized queries via JPA.

3. Methodology

3.1 Methodological Approach

Three practices converged for this. The first, **SDD (Software Design Documentation)**: before starting to structure the code, the design must be documented and kept alive with every modification; it evolves with the system and is not archived. The second, DDD: the business model was built before the code, with a map of contexts, entities, invariants and domain events. The third, Scrum: a methodology considered an agile management framework, based on user stories, which were written in the Given/When/Then format, with story points, acceptance criteria and a checklist shared by the team (DoD). In addition to a general habit: every relevant architectural decision was recorded as an ADR (Architecture Decision Record), so that there is a record of which alternatives were evaluated, which were chosen and which were discarded.

3.2 System Architecture

RentaMovil works through client-server communication and an API Gateway as the single entry point and as the communication channel between services through REST contracts. The clients — the customer web app, the administrative web app and the mobile app — do not talk to the services directly: they go through the Gateway, which routes each request by path to the corresponding microservice, validates the JWT token and applies rate limits. Each microservice implements its own hexagonal architecture, which decouples its business logic from the details of its infrastructure. The relational database is a single one at the physical level, but each microservice owns its own schema, clearly delimited with its respective access rules.

Figure 1 summarizes the referenced architecture.

3.3 Domains and Decomposition into Services

The domain is split into seven contexts. Now then, a context does not have to be equivalent to a deployable service, for deployment economy, and also because not all contexts justify their own deployment process. For that reason we group those contexts that share a life cycle.

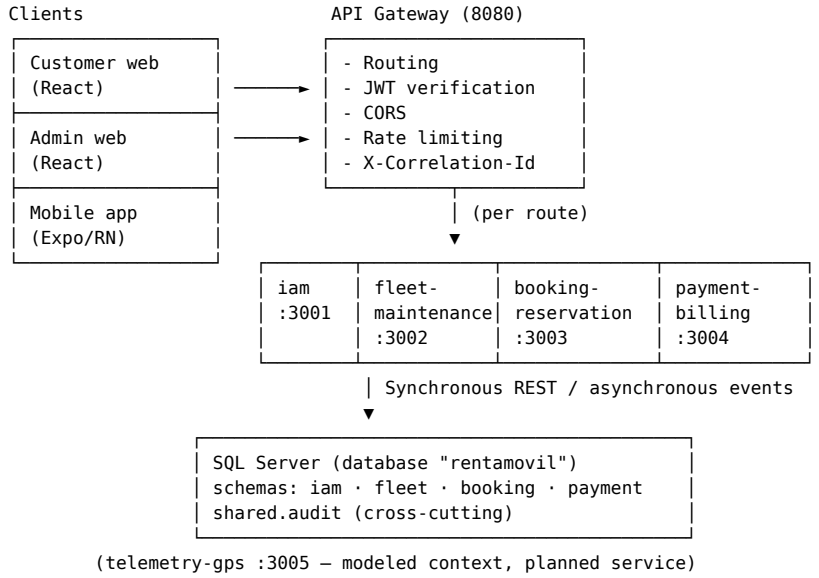


Figure 1. RentaMovil reference architecture: microservices, API Gateway and schema per service.

Reservations, rental execution and notification, for example, live together. A reservation and its rental are the same transaction with two states. Separating them would only have added network calls with no gain.

Table 1 organizes the relationship between contexts, services and tables.

3.4 Technology Selection

The technologies were chosen for maturity, performance and, of course, for their remarkable ease of maintenance. The backend ended up in Java 21 with Spring Boot: a proven infrastructure, with dependencies, as well as security and data access with factory defaults. Is it necessary to unify the versions of all the services? Of course not; each one is independent and pins its own versions, just as happens with the Gateway. Data is stored in a normalized form in Microsoft SQL Server, using Liquibase to handle database versioning. The quality of the database is sustained with foreign keys, CHECK constraints and application users with permissions granted within the schema itself.

On the frontend, React with Vite was used for the web (a component model and a fairly broad ecosystem), and Expo + React Native, which allows sharing styles across iOS, Android and web. For local deployment, Docker and Docker Compose were used, based on a network where services find each other by name.

3.5 Applied Design Patterns

3.5.1 Ports and Adapters

Each service separates three zones: domain (entities, invariants, ports), application/usecase (the use cases) and infrastructure (persistence, security, web and client adapters). The domain knows nothing about Spring or JPA. To change the database, the broker or the REST client, nothing is modified other than an adapter; the business rules are not even aware of the change.

3.5.2 Repository and Service Layer

Data access is achieved through repositories (Spring Data JPA interfaces) behind output ports. Meanwhile, the logic lives in the services and use cases; the controllers contain no rules. Being able to test with doubles (mocks, fakes) becomes a benefit, since these tests are simple and the code is reused without dragging in or affecting dependencies.

3.5.3 Data Transfer Object (DTO)

The controllers speak in input and output DTOs. Neither the domain entities nor the JPA ones show through the API. This way we control what is exposed, we validate what comes in, and we can rework the internal model without breaking the client-facing contract.

3.5.4 Domain Event Publisher and Idempotent Consumer

The services publish domain events such as ReservationCreated and RentalCompleted, and those events are consumed by other internal endpoints. Delivery, as is well known, can be repeated. That is why the consumers are idempotent; having to process the same event twice does not alter the system, since it leaves it the same as having processed it only once.

3.5.5 API Gateway and Schema per Service

The Gateway unifies routing and edge security. What defines who owns each table is the schema of each service, which also makes it possible to isolate database permissions per service (we documented this in ADR009).

3.6 Implemented Security

Security was designed in layers. Authentication is delegated to JWT tokens signed with RS256; the iam service is the one in charge of keeping the private key and publishes the public key at GET /jwks; the other services verify the signature with that key, so that no one other than iam can issue tokens. The access token lasts only 30 minutes and the refresh token 7 days, with single-use rota-

#	Context/Service	Responsibility	Port	Own tables
01	Api Gateway	Single entry point: routing, authentication, limits	8080	-
02	Iam	Registration, login, JWT tokens, roles and permissions	3001	Person, user, role, permission, role_permission, verification_code, session
03	Fleet-maintenance	Vehicle catalog, branches, maintenance	3002	Branch, Branch_operating_hour, Brand, category, engine_type, vehicle_model, vehicle, maintenance_type, vehicle_maintenance
04	Booking-reservation	Reservation, rental execution, notifications and insurance	3003	Insurance_type, reservation, rental, notification
05	Payment-billing	Payments by transfer, receipt review, invoices	3004	Bank_account, payment, invoice
06	Telemetry-gps	Location capture and route history	3005	Gps, location (planned)

Table 1. Bounded contexts, deployable services and own tables.

tion. Thus, if an already consumed refresh token arrives, all of the user's sessions are brought down. Authorization is fine-grained, with permissions in the entity:action format, for example roles:grant, and three fixed roles, which are ADMIN, CLIENT and SUPERADMIN. If five failed login attempts are made, the account is locked; only a superadministrator unlocks it.

3.7 Metrics System (KPIs)

To ensure that we measure the engineering quality of our software adequately, we defined ten indicators with industry targets clearly adjusted to what a training project can actually sustain. To avoid subjective objections, such as the software being slow or unreliable, the indicators cover four dimensions: performance (latency of critical endpoints), reliability (target availability and recovery from failures), code quality (test coverage, domain coupling, security) and process (traceability, documentation, idempotency).

3.8 Applied Good Practices

To comply with the ideas of clean code [Martin 2008], short functions with a single responsibility were applied, and comments only when they add context. We were rigorous and respected the SOLID principles, especially applying dependency inversion and single responsibility. To protect data integrity, several fronts are protected at once: validation in the client form, DTO validation with Bean Validation in the controller, business rules in the use case, and database constraints as the last barrier. For code versioning and collaboration: branch conventions (feat/, fix/), Pull Request reviews with Squash and Merge, and a shared definition of done.

4. Implementation

4.1 Common Backend Structure

All services share the same hexagonal skeleton: domain/ for entities, enumerations, exceptions and input and output ports; application/usecase/ with each use case; and infrastructure/ with configuration, persistence, security, external clients and the web layer. A rule that reinforces this is that the service never touches the schema (spring.jpa.hibernate.ddl-auto=validate); it only checks that the entities match the fields of the tables created by the migrations. Any schema change begins with a changeset in the corresponding database repository.

4.2 IAM Service (Identity and Access)

The iam service enables customer registration, login, token renewal, password recovery, profile management and the administration of roles and permissions. It exposes POST /register, POST /login, POST /refresh, POST /password/forgot, PATCH /users/me and the administration operations PATCH /users/{id}/role and PATCH /users/{id}/status. When the service starts, the JWT keys are validated: if any is invalid or has fewer than 2048 bits, the system does not start. For security, the first superadministrator is created manually through an independent process. This way we avoid leaving passwords or hashes exposed in the repository and guarantee that session credentials are not identifiable in plain text.

4.3 Fleet & Maintenance Service

Thanks to this service, it is determined which vehicles exist, and their status and location are controlled. Vehicles and branches are managed, including the operating hours of the latter. Brands, models, categories and engine types are also managed, in addition to the maintenance cycle. Vehicles have four states: AVAILABLE, RENTED, MAINTENANCE and RETIRED. Their availability is not stored; it is queried. Two business rules: an occupied vehicle does not move to maintenance, and RETIRED is terminal, because rows are never physically deleted.

4.4 Booking & Reservation Service (Reservations and Rentals)

The life cycle of each reservation is defined: it is created, queried, modified (only the return branch), cancelled and left to expire on its own; in addition to recording the delivery and return of the vehicle and internal notifications. The states of each reservation are: PENDING_PAYMENT → PENDING_REVIEW → CONFIRMED → COMPLETED, with exits to CANCELLED. The totals are the sum of the vehicle subtotal

and the insurance. The business rules are concentrated solely in the domain. For example, the return branch can only be modified with three days' notice before the reservation ends. A reservation without a payment receipt is cancelled when the 24-hour deadline expires.

4.5 Payment & Billing Service (Payments and Invoicing)

The payment system applied in RentaMovil is not a payment gateway, owing to how the project is oriented. Instead, payment by transfer was implemented. The customer transfers the exact value of the reservation to one of the company's accounts and uploads the receipt; the administrator reviews it and approves or rejects it. The service is in charge of verifying that the amount matches the total value of the reservation; partial payments are not allowed. With this service, the SUPERADMIN role can manage all bank accounts, and also issues the invoice with sequential numbering. And to maintain consistency with the schema-per-service approach, the amount is requested through the booking-reservation API; a direct JOIN with its table is never performed.

4.6 Telemetry & GPS Service (Telemetry)

It tracks the vehicle while it is rented and keeps the route history. It is the one that writes the most: a device reports positions nonstop during the rental. Because of this, it is the perfect candidate for a time-series database if the volume ever justifies it.

4.7 API Gateway

The Gateway is the only component exposed outside the internal network. It routes by path, /auth/ to iam, and in addition to verifying the signature and validity of the token, it injects the user and role identifiers into the services, applies CORS, adds an X-Correlation-Id to each request so it can be traced in the logs, and applies rate limits. If a service does not respond, it returns 503; if the timeout is exceeded, 504. For added security, a design detail was added: each service verifies the token again before authorizing any operation.

4.8 Persistence and Migration Strategy

To avoid separate databases, the rentamovil database is a single one at the physical level, but organized into schemas per service: iam, fleet, booking and payment, plus shared for auditing. To achieve this, each service has its own migrations repository (rtm-<service>-db) with Liquibase and its own change history. In each repository, an order by statement type was set: 01_ddl (schemas, tables without foreign keys, then foreign keys and indexes), 02_dml, 03_dcl (database users and permissions), 04_tcl and 05_rollbacks. Between repositories, the order is iam → fleet-maintenance → telemetry-gps → booking-reservation → payment-billing, so that foreign keys between services always point to existing tables. The service connects with a user restricted to its own schema so that data ownership and the shared database stop contradicting each other.

4.9 Web Frontend

The web app was built with React and Vite, using a feature-first architecture with its organization by functionality and component architecture. Each feature (authentication, catalog, reservations, payments, administration, reports) brings together components, hooks, services, utils and pages; whatever is reusable lives in a shared directory. To avoid delays on the frontend side while the backend is being built, we built a mock data server. The interface includes internationalization, validated forms, maps for location and charts for the administrative dashboards.

4.10 Mobile Application

To build the mobile application, Expo and React Native were used, with Expo Router for navigation and Zustand for global state. It is typed with TypeScript and organized by features, just like the frontend. Navigation does not know the business logic, and only the services can encapsulate the API calls.

4.11 Request Flow

Let us follow the journey of a request. The user taps an element in the frontend and the client sends an HTTP request with the access token. The Gateway validates the token and the route, then forwards to the assigned service with a correlation identifier. The service controller repeats the token verification, validates the input DTO and delegates to the use case. The business rules are applied and the output ports are used to read or write. The request climbs back up and becomes a consistent JSON. Errors are also uniform: code, message and validation detail.

4.12 Communication between Services and Domain Events

Communication is synchronous when immediate consistency is needed, such as checking the availability of a vehicle before creating the reservation. Asynchronous communication is applied when the system's response can wait, such as generating a notification when a payment is confirmed. Each domain event is published with a common envelope and consumed at internal endpoints that do not leave the network. Table 2 summarizes the main events and what triggers them.

5. Testing and Validation

5.1 Testing Strategy

The strategy has the shape of a pyramid: unit tests at the base, contract and integration tests in the middle, and a few end-to-end tests at the top [Cohn 2009; Fowler 2012]. The target proportion is 50% unit, 20% contract, 25% integration and 5% end-to-end. That distribution keeps integration stable and, likewise, avoids the antipattern of turning the pyramid upside down. We do not give coverage percentages because JaCoCo [JaCoCo n.d.] is not yet configured, so we report how many tests were run and how many passed. The repositories are independent Maven

Event	Published by	Consumed by / effect
ReservationCreated	booking-reservation	Notification to the administrator
PaymentCompleted	payment-billing	Reservation moves to CONFIRMED, notification to the customer
PaymentRejected	payment-billing	Reservation returns to PENDING_PAYMENT or is cancelled, depending on the outcome
InvoiceGenerated	payment-billing	Notification with the invoice download link
RentalStarted	booking-reservation	Fleet-maintenance marks the vehicle as RENTED; telemetry starts tracking
RentalCompleted	booking-reservation	Fleet-maintenance marks the vehicle as AVAILABLE; telemetry stops tracking
MaintenanceCompleted	fleet-maintenance	booking-reservation creates the corresponding notification

Table 2. Main domain events.

projects, with no aggregator POM. Each module is tested separately with JDK 21 and mvn test.

5.3 Backend Tests

Each unit test is performed with JUnit 5, Mockito and AssertJ [Meszaros 2007]. The use cases are isolated from Spring and external dependencies are replaced with test doubles. Of greatest importance are the business rules, as well as the errors and relevant interactions. Table 3 shows the baseline executed on October 9, 2026.

The 225 tests add up across five Maven runs, not a single multi-module run. Billing is the module with the fewest tests.

5.3 Persistence, Controller and Contract Tests

Fleet has a test suite with @DataJpaTest on H2. It covers the queries for branches, vehicles, maintenance and models. H2 helps validate the behavior of JPA and JPQL, but it does not replace SQL Server. The shared.audit schema is left out, since it does not exist in H2 and causes Hibernate to log a warning. IAM, Booking and Billing use JPA in production.

Some tests treat the filters as isolated units and others call the controllers directly. This is useful for testing delegation and local logic. They do not go through the MVC server, so HTTP routes are not verified, nor is JSON serialization included, nor is input validated. In the five modules, no tests with @WebMvcTest or MockMvc are performed.

Each service has an OpenAPI contract [OpenAPI Initiative n.d.], which serves as documentation and as an agreement between producer and consumer. There are not yet any automated tests that verify that the implementation complies with it.

5.4 Frontend Tests

The automated baseline covers only the backend. While the frontend was being developed, so as not to hinder structural changes, a mock data server was created, which made it possible to reproduce errors and empty states without depending on the services. There is no automated test suite for the web or mobile app.

5.5 Integration and End-to-End Tests

The integration evidence was validated manually, so it is kept separate from the automated evidence. On October 4, a verification was carried out with Docker, SQL Server, the Gateway and Postman, with 37 of 37 requests successful. With the system already deployed, on October 9 the Gateway health endpoint responded ok with the four services. The web frontend loaded with HTTP 200, and the team walked through the flow of the installed mobile app against the public backend.

Thanks to that manual integration, ten corrective findings (C-01 to C-10) were made, which the automated tests had not detected. Several errors arose between what the client sent and what the backend expected, such as different field names, wrongly assumed routes, and a rotated refresh token that the client did not store, which caused a login loop. Those failures would have been detected by contract tests and MockMvc tests. The critical flows that were walked through were the creation of a reservation and the review of the payment receipt, since this is not automated.

5.6 Security Tests

A full OWASP audit was not performed. Some controls are backed by automated tests, and others exist only as design decisions. Table 4 separates them.

5.7 Traceability

The rentamovil project maintains a matrix that connects each functional requirement with its user story and its service. There are 31 stories in seven epics, each with verifiable acceptance criteria. The tests column of that matrix was filled in before implementation and is not yet populated with the real tests. The matrix marks 32 rows as pending and one in progress. Bringing it up to date with the 33 classes that exist is an open improvement action.

6. Results

6.1 Product Built

A distributed system was built with four business services and an API Gateway, with two clients (web, mobile) and a shared relational database with one schema per service. Table 5 gathers the technical metrics drawn from the repositories.

6.2 Quality Indicators

Ten indicators were used to carry out the evaluation with the evidence available at the close of the analyzed phase. Where there was no evidence, the indicator was left unmeasured.

Four indicators met, three partial and three not measured. If each is worth 1, 0.5 or 0 points, the engineering

Module	Behavior exercised	Tests	Failures/Errors
iam	Registration, login, logout, token refresh, profile, email, password, roles and status	34	0 / 0
fleet-maintenance	Vehicles, availability, maintenance, branches, rental events, JWT verification and JPA queries with H2	114	0 / 0
booking-reservation	Reservations, rentals and insurance, payment events, reservation expiration	53	0 / 0
payment-billing	Payment entity and payment use case	14	0 / 0
api-gateway	JWT filter, correlation ID and rate limiter	10	0 / 0
Total	33 test classes	**225**	**0 / 0**

Table 3. Results of the automated backend tests.

Control	Evidence
JWT verification at the edge and in the services	Automated: JwtAuthenticationFilterTest (gateway) and JwksTokenVerifierAdapterTest (Fleet)
Per-IP request limit	Automated: InMemoryRateLimiterTest
Correlation identifier	Automated: CorrelationIdFilterTest
Login, logout, refresh, password, roles and user status	Automated: IAM use cases
Restricted CORS, BCrypt, parameterized queries, secrets in environment variables, minimum permissions per schema	Design decision, no automated test

Table 4. Security controls and their evidence.

Metric	Value
Business microservices + gateway	4 + 1 (telemetry modeled, not implemented)
Bounded contexts modeled	7
Domain tables (schemas iam, fleet, booking, payment)	23 (+ cross-cutting audit)
Production Java files (backend)	399
Test classes / automated tests	33 / 225
Documented REST endpoints	More than 60
User stories	31 in 7 epics
OpenAPI contracts	7 (one per service, including telemetry, plus one shared contract)
Architecture decision records (ADR)	9
Web / mobile client files	305 / 146
Corrective, preventive and improvement actions recorded	10 / 5 / 5

Table 5. Technical metrics of the project.

#	Indicator	Target	Status	Evidence
1	Test coverage	≥ 80 %	Not measured	JaCoCo not configured; 225 tests pass
2	p95 latency on critical endpoints	< 2 s	Not measured	Manual validation; no load tests (NFR-001)
3	Availability	≥ 99 % monthly	Pending	No real production (NFR-002)
4	Security against OWASP Top 10	0 critical findings	Partial	Controls implemented and partly tested; no audit
5	Portability (containers)	100 % of services	Met	Dockerfile in all five modules
6	Domain coupling	0 framework dependencies	Met	domain package without Spring or JPA
7	Traceability requirement → story → test	100 %	Partial	Requirement → story → service complete; tests column pending update
8	Idempotency of event consumers	100 % of events	Met	Consumers idempotent by state
9	Documentation per service	100 % with README and contract	Met	Seven OpenAPI contracts and rtmdocs
10	MVP functional coverage	≥ 80 %	In progress	Four of five services operational

Table 6. Dashboard of system quality indicators.

maturity index comes to 55 out of 100. That number tells us how many of the practices the team set out to follow were fulfilled. It says nothing about production performance.

6.3 Functional Compliance

On October 9, 2026, the platform was deployed with Docker, nginx and ngrok. All the real services responded through the Gateway, the web frontend loaded, and the mobile app was installed as an APK and likewise tested

against the public backend. Identity, fleet, reservations and payments work with one another, and the clients consume their contracts. The telemetry context remains unimplemented; for that reason the frontend does not reflect the data sent by the GPS. The main flow comprises browsing the catalog, checking availability, creating a reservation, uploading the receipt, approving it, confirming the reservation, and recording the delivery and return. The user stories cover it, as do the domain rules tested in Booking.

7. Discussion

7.1 Impact of Hexagonal Architecture and Microservices

By implementing hexagonal architecture in each microservice, a notable improvement in code maintainability was achieved. By separating the what, in the domain; the how, in the application; and the what-it-communicates-with, in the infrastructure, it is made clear where each change must be made. A business rule lives in the domain, and a technology change becomes an adapter. Likewise, the boundary between services was set by the domain and not by the screens, so as to prevent the interface from ending up dictating the architecture. What is shared stayed with the API Gateway (perimeter authentication, CORS, limits and correlation), so each service can concern itself with its own matters.

7.2 Analysis of the KPIs

The indicators that best show the value of the design are domain coupling, portability and documentation, since they are verified statically. An isolated domain is tested without altering anything else, and a containerized service is deployed without major problems. The tools to measure coverage, latency and availability have not yet been applied. Traceability was left incomplete since, even with the existing matrix, the tests were not updated.

7.3 Architectural Decisions and Trade-offs

The important decisions were recorded as ADRs, together with the alternatives that were discarded. The first decision was to use one shared database with one schema per service, instead of complicating ourselves with one database per service. This gains referential integrity across contexts and avoids distributed transactions, at the cost of coordinating the order of the migrations. The second was to query a vehicle's availability synchronously instead of maintaining a local copy updated from events. A small coupling was chosen over including an inconsistency such as the customer being able to query a vehicle in a reserved state. The third was to structure contexts with the same life cycle into a single service. We avoided the network coordination overhead that takes away the project's scalability. An efficient architecture is born of explicit, well-founded trade-offs, not of the systematic application of patterns without sense or coherence.

7.4 Value of Documentation as Part of the Design

Using the SDD approach was of great importance: documenting before implementing made it possible to discuss the boundaries and scope between services and the API contracts when changing them still cost little. Defining the business rules as fixed rules and the stories with verifiable criteria reduced inconsistencies and contradictions at coding time. Having the documentation as the source of truth while the code was being developed, with standardized names, templates and cross-references, avoided ambiguity.

7.5 Lessons Learned

Defining the architecture from the start saves time and resources and is more efficient than refactoring everything afterward. It was also learned that, by documenting and having foundations, one verifies before assuming. Several errors and inconsistencies that arose were the lack of standardization, invented data, or inconsistency in field names. Idempotency has to be designed from the start if one is going to work with events, since delivery can be repeated. Granular security becomes indispensable, even if at first it complicates startup, but it is of utmost importance when customers, administrators and superadministrators coexist. In addition, defining an explicit API contract is the best way for clients and services to evolve together.

The architecture was refactored several times due to the lack of planning in the documentation. It was also learned that unit tests do not replace contract validation. Errors such as refresh tokens not being stored and misaligned routes. At times, people worked with different versions of the code, causing conflicts in the shared work. Carrying out the findings through a cycle of corrective, preventive and improvement actions in accordance with the ISO/IEC 29110 standard [ISO/IEC 2018] makes explicit what failed, why, and how the solution was found. Even so, one experience was learning to pin the Java version value, since because of it a run with JDK 27 failed when initializing Lombok, as the baseline is only produced with JDK 21.

7.6 Relation to Previous Work

The results agree with what Newman [2021] and Fowler and Lewis [2014] said regarding autonomy and functional cohesion. The usefulness of the API Gateway and of the idempotent consumer is that described by Richardson [2018]. The established separation between domain and infrastructure is an explicit implementation of hexagonal architecture [Cockburn 2005] and of dependency inversion [Martin 2017]. The indicator dashboard rests, with adaptations, on the DevOps measurement culture [Forsgren et al. 2018; Humble and Farley 2010]. A continuous integration pipeline is an indispensable requirement for automating the collection of the DORA metrics.

8. Conclusions

8.1 Summary of Findings

Even in an academic context such as the one in which RentaMovil was built, applying DDD and hexagonal architecture and combining it with microservices resulted in a scalable system, with high maintainability and, above all, easy to understand. The result of these implementations was a platform with four business services and a Gateway, 225 automated tests run without failures, in addition to a verified deployment and a documented manual integration. Defining and separating the domain from the application and the infrastructure left responsibilities clear and well defined. Thanks to the schema per service, it was made clear who owned the data, and the Gateway took charge of specifying what is cross-cutting.

8.3 Limitations

Because it is an academic project, the resource limitations are quite clear, in addition to not having a production infrastructure; therefore, latency could not be measured under real conditions. The tests lack contract tests and HTTP tests with MockMvc. SQL Server runs as a single instance, without high availability (risk R-003). The Gateway's rate limiter works in memory and the backend URL is fixed inside the APK. Lastly, the volume of users is low, but we hope to explore this in a high-concurrency context.

8.4 Future Work

An order is proposed: improve and clarify the current business rules. Add MockMvc and contract tests, as well as persistence tests for IAM, Booking and Billing, with Testcontainers [Testcontainers n.d.] to have real compatibility with SQL Server. In addition, fix errors in the telemetry context due to the missing tests that depend on the GPS. Replace the in-memory event channel with a broker with delivery guarantees. Implement a continuous deployment pipeline in production environments. Likewise, if the project were to reach production, the gradual move to one database per service would have to be evaluated.

8.5 Final Reflection

RentaMovil turns a manual process into a platform organized around the domain, with documented decisions and established boundaries. Clarifying, defining and establishing the architecture and business rules in the documentation allowed a better evolution in the development of the system, thus avoiding ambiguities as well as contradictions arising from shared work, with the documentation as the single source of truth. Thanks to the quality metrics, the system can be explained, tested, and one can know exactly what could be improved or strengthened. The quality of a product does not appear on its own; it is sustained by planning, design and perseverance.

Appendix A. Reproducibility Checklist

Data. Normalized relational model in Microsoft SQL Server, rentamovil database, with reference data seeded by migrations and synthetic data for tests.

Code. Independent repositories per service and per database, documented hexagonal architecture, branch conventions and Pull Request review.

Environment. JDK 21, Maven 3.9.16, SQL Server in Docker, shared Docker network rentamovil, Node.js for the clients and Liquibase for the migrations.

Procedure. Bring up the databases and apply the migrations in the documented order, start the services and the Gateway, authenticate with iam and walk through the critical flows through the Gateway. For the automated tests, run mvn -f <module>/pom.xml test in each of the five modules. The reports are left in target/surefire-reports.

Results. Quality indicators, OpenAPI contracts, user stories and traceability matrices available in the documentation repository.

References

AssertJ. n.d. AssertJ – Fluent assertions for Java. https://assertj.github.io/doc/

Alistair Cockburn. 2005. Hexagonal Architecture (Ports and Adapters). https://alistair.cockburn.us/hexagonal-architecture/

Mike Cohn. 2009. Succeeding with Agile: Software Development Using Scrum. Addison-Wesley.

R. D. C. Espinosa and J. C. C. Eraso. 2024. Gestión de tecnología y buenas prácticas en una empresa de desarrollo de software colombiana. Revista de los 17, 62 (2024).

Eric Evans. 2003. Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley.

Nicole Forsgren. 2021. DORA Metrics: The Four Keys to Software Delivery Performance. Google Cloud. https://dora.dev/

Nicole Forsgren, Jez Humble, and Gene Kim. 2018. Accelerate: The Science of Lean Software and DevOps. IT Revolution Press.

Martin Fowler. 2002. Patterns of Enterprise Application Architecture. Addison-Wesley.

Martin Fowler. 2012. TestPyramid. https://martinfowler.com/bliki/TestPyramid.html

Martin Fowler and James Lewis. 2014. Microservices. https://martinfowler.com/articles/microservices.html

Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. 1994. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Jez Humble and David Farley. 2010. Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley.

ISO/IEC. 2018. ISO/IEC 29110-4-1:2018, Systems and software engineering — Lifecycle profiles for Very Small Entities (VSEs) — Part 4-1: Software engineering — Profile specifications: Generic profile group. International Organization for Standardization.

JaCoCo. n.d. JaCoCo Java Code Coverage Library. https://www.jacoco.org/jacoco/

Michael B. Jones, John Bradley, and Nat Sakimura. 2015. JSON Web Token (JWT). RFC 7519. Internet Engineering

Task Force. https://www.rfc-editor.org/rfc/rfc7519
JUnit. n.d. JUnit 5 User Guide. https://junit.org/junit5/docs/current/user-guide/
Liquibase. n.d. Liquibase Documentation. https://docs.liquibase.com/
Robert C. Martin. 2008. Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
Robert C. Martin. 2017. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Prentice Hall.
Gerard Meszaros. 2007. xUnit Test Patterns: Refactoring Test Code. Addison-Wesley.
Mockito. n.d. Mockito – Mocking framework for unit tests in Java. https://site.mockito.org/
Sam Newman. 2021. Building Microservices: Designing Fine-Grained Systems (2nd ed.). O'Reilly Media.
Michael Nygard. 2011. Documenting Architecture Decisions. Cognitect Blog. https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions
OpenAPI Initiative. n.d. OpenAPI Specification. https://spec.openapis.org/oas/latest.html
OWASP Foundation. 2021. OWASP Top 10: The Ten Most Critical Web Application Security Risks. https://owasp.org/Top10/
David L. Parnas. 1972. On the Criteria To Be Used in Decomposing Systems into Modules. Communications of the ACM 15, 12 (1972), 1053–1058.
Chris Richardson. 2018. Microservices Patterns: With Examples in Java. Manning.
Ken Schwaber and Jeff Sutherland. 2020. The Scrum Guide. https://scrumguides.org/
Ian Sommerville. 2015. Software Engineering (10th ed.). Pearson.
Spring. n.d. Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/
Testcontainers. n.d. Testcontainers – Integration tests with real dependencies. https://testcontainers.com/